Building xG: Dataset Creation and Regression Analysis

2025-12-09

Background

In recent years a particular soccer statistic, Expected Goals (xG), has grown in popularity. This has caused much discourse about its accuracy, relevancy, and usefulness in relation to the statistic's ability to capture the quality of a shot. Possible xG values range from 0 to 1, so, at its core, xG is the expected value of a shot. If a particular shot was taken ten times and you would expect to score seven of the ten, the shot would be assigned an xG of 0.7. But how is xG calculated professionally and how can it be recreated? This project answers these questions.

xG can be calculated using classical statistical techniques like regression and penalized regression, or through use of machine learning models like Extreme Gradient Boosting (XGBoost). Applying these models requires a data set and so that is where I will begin. From there, using the different types of analysis, distinct models can by built and used to estimate xG. This process is being conducted on a a dataset provided by Hudl Statsbomb and so the efficacy of my models can be compared to Hudl's to check how well my models' predictions align with theirs.

Data Set Creation and Modification

In order to understand how the dataset that I will use is created, there is some necessary background information about how the data is accessed. I am using freely available, online data. I am using "event" data which is grouped by match and then by competition. Thus, to generate the complete dataset, it is necessary to look within each grouping and bind all of the available data. Since this project looks specifically at shots, all the non-shot events will be filtered out. Furthermore, all statistics that are non shot-related will be filtered. This will eventually leave a dataset of roughly 80000 observations of about 20 different variables.

Helper Functions

Building this dataset begins with the creation of some helper functions. These helper functions serve a variety of purposes. Some clean specific statistics into usable data while others generate more statistics based on given data. There is one helper function that serves to collect all of this information and place it into a usable dataset.

Assist Type

This helper function is used to look at metrics describing the assist, if there is one. Given a specific metric and the id of the corresponding assist, it returns "TRUE" or "FALSE".

```
getAssistInfo <- function(pid, events, type) {
  if (is.na(pid)) {
    return("Unassisted")
  }
```

```
  assist <- events |>
    filter(id == pid)

  if (!type %in% colnames(assist)) {
    return("FALSE")
  }
  assist <- assist |> select(type)
  if (!is.na(assist[1]) && assist[1] == TRUE) {
     return("TRUE")
  }

  return("FALSE")
}
```

Obstructing Players

This helper function accesses a data frame of every available player location in order to check whether that player is obstructing the balls path to goal. The formula used is dependent on the location of the ball in relation to the goal. Once it is determined which formula to use, the function calculates and returns the number of non-goalkeeper players in between the ball and the goal.

```
getClarity <- function(shot_loc, df) {
  if (!is.data.frame(df)) {
    return(NA)
  }
  df <- df |> as.data.frame()
  if (shot_loc[2] != 36) {
    slope1 = (shot_loc[1]-120)/(shot_loc[2]-36)
  }
  if (shot_loc[2] != 44) {
    slope2 = (shot_loc[1]-120)/(shot_loc[2]-44)
  }
  df <- df |> filter(position.id != 1)
  df <- df |> rowwise() |> mutate(
    obstructing = case_when(
      shot_loc[2] < 36 &&
        location[1] < slope1*(location[2]-shot_loc[2])+shot_loc[1] &&
        location[1] > slope2*(location[2]-shot_loc[2])+shot_loc[1] ~ TRUE,
      36 < shot_loc[2] && shot_loc[2] < 44 &&
        location[1] > slope1*(location[2]-shot_loc[2])+shot_loc[1] &&
        location[1] > slope2*(location[2]-shot_loc[2])+shot_loc[1] ~ TRUE,
      44 < shot_loc[2] &&
        location[1] > slope1*(location[2]-shot_loc[2])+shot_loc[1] &&
        location[1] < slope2*(location[2]-shot_loc[2])+shot_loc[1] ~ TRUE,
      shot_loc[2] == 36 &&
        location[2] > 36 &&
        location[1] > slope2*(location[2]-shot_loc[2])+shot_loc[1] ~ TRUE,
      shot_loc[2] == 44 &&
        location[1] > slope1*(location[2]-shot_loc[2])+shot_loc[1] &&
        location[2] < 44 ~ TRUE,
      .default = FALSE
    )
```

```
  )
  return(sum(df$obstructing))
}
```

Goalkeeper Stats

GK Positioning This statistic uses the same logic and process as the function to determine the number of obstructing players. The difference is that this function is specifically focusing on whether or not the opposing goalkeeper is obstructing the path of the ball.

```
getGKpos <- function(shot_loc, df) {
  if (!is.data.frame(df)) {
    return(NA)
  }
  df <- df |> as.data.frame()
  if (shot_loc[2] != 36) {
    slope1 = (shot_loc[1]-120)/(shot_loc[2]-36)
  }
  if (shot_loc[2] != 44) {
    slope2 = (shot_loc[1]-120)/(shot_loc[2]-44)
  }
  df <- df |> filter(position.id == 1 & teammate == FALSE)
  if (nrow(df) == 0) {
    return(FALSE)
  }
  df <- df |> rowwise() |> mutate(
    obstructing = case_when(
      shot_loc[2] < 36 &&
        location[1] < slope1*(location[2]-shot_loc[2])+shot_loc[1] &&
        location[1] > slope2*(location[2]-shot_loc[2])+shot_loc[1] ~ TRUE,
      36 < shot_loc[2] && shot_loc[2] < 44 &&
        location[1] > slope1*(location[2]-shot_loc[2])+shot_loc[1] &&
        location[1] > slope2*(location[2]-shot_loc[2])+shot_loc[1] ~ TRUE,
      44 < shot_loc[2] &&
        location[1] > slope1*(location[2]-shot_loc[2])+shot_loc[1] &&
        location[1] < slope2*(location[2]-shot_loc[2])+shot_loc[1] ~ TRUE,
      shot_loc[2] == 36 &&
        location[2] > 36 &&
        location[1] > slope2*(location[2]-shot_loc[2])+shot_loc[1] ~ TRUE,
      shot_loc[2] == 44 &&
        location[1] > slope1*(location[2]-shot_loc[2])+shot_loc[1] &&
        location[2] < 44 ~ TRUE,
      .default = FALSE
    )
  )
  return(df$obstructing)
}
```

GK Distance This function looks at the distance from the goalkeeper to the ball at the time of the shot.

```r
getGKdist <- function(shot_loc, df) {
  if (!is.data.frame(df)) {
    return(NA)
  }
  df <- df |> as.data.frame()
  df <- df |> filter(position.id == 1 & teammate == FALSE)
  if (nrow(df) == 0) {
    return(NA)
  }
  df <- df |> rowwise() |> mutate(
    x = location[1],
    y = location[2],
    .after = location
  )
  l <- (shot_loc[1] - df$x[1])^2
  h <- (shot_loc[2] - df$y[1])^2
  return(sqrt(l+h))
}
```

Filter Shot data

All the previously described functions are used here to filter and clean all of data surrounding shots for a given match. This returns a data frame that can be used to run the necessary analysis processes.

```r
getnewShots <- function(m) {
  events <- get.matchFree(m)

  #make df with all shot events and relevant data
  shots <- events |> filter(type.id == 16 & shot.type.id != 88) |>
    select(c("location", "under_pressure", "play_pattern.name",
             starts_with("shot.")))
  shots <- shots |> select(-ends_with(".id"), -shot.end_location)

  #mutate table to have independent distance and angle variables
  shots <- shots |>
    rowwise() |>
    mutate(
    "distance" = 120 - location[1],
    "angle" =
      case_when(
        location[2] >= 40 ~ atan2(distance, location[2]-40)*180/pi,
        location[2] < 40 ~ 180 - atan2(distance, location[2]-40)*180/pi
      ),
    .after = "location"
  )

  #make a goal/no goal variable
  shots <- shots |>
    mutate(
      outcome =
        case_when(
          shot.outcome.name == "Goal" ~ 1,
          .default = 0
```

```r
      )
  )

#NAs represent false in some columns
#Change all NAs to FALSE
if (!"under_pressure" %in% colnames(shots)) {
  shots <- shots |> mutate(under_pressure = NA)
}
if (!"shot.first_time" %in% colnames(shots)) {
  shots <- shots |> mutate(shot.first_time = NA)
}
if (!"shot.aerial_won" %in% colnames(shots)) {
  shots <- shots |> mutate(shot.aerial_won = NA)
}
if (!"shot.one_on_one" %in% colnames(shots)) {
  shots <- shots |> mutate(shot.one_on_one = NA)
}
shots <- shots |>
  mutate(
    pressure = case_when(under_pressure == TRUE ~ TRUE, .default = FALSE),
    first_time = case_when(shot.first_time == TRUE ~ TRUE, .default = FALSE),
    aerial_won = case_when(shot.aerial_won == TRUE ~ TRUE, .default = FALSE),
    one_on_one = case_when(shot.one_on_one == TRUE ~ TRUE, .default = FALSE),
    .after = shot.key_pass_id
  )

shots <- shots |> rowwise() |> mutate(

  assist.aerial = getAssistInfo(shot.key_pass_id, events, "pass.aerial_won"),
  assist.cross = getAssistInfo(shot.key_pass_id, events, "pass.cross"),
  assist.switch = getAssistInfo(shot.key_pass_id, events, "pass.switch"),
  assist.through = getAssistInfo(shot.key_pass_id, events, "pass.through_ball"),
  assist.cut_back = getAssistInfo(shot.key_pass_id, events, "pass.cut_back"),
  assist.inswinging = getAssistInfo(shot.key_pass_id, events, "pass.inswinging"),
  assist.outswinging = getAssistInfo(shot.key_pass_id, events, "pass.outswinging"),
  assist.deflected = getAssistInfo(shot.key_pass_id, events, "pass.deflected"),

  obstruction = getClarity(location, shot.freeze_frame),

  GKpos = getGKpos(location, shot.freeze_frame),
  GKdist = getGKdist(location, shot.freeze_frame),
  .after = shot.freeze_frame
)

#remove location variable and freezeframe/assist_id
shots <- shots |>
  select(distance, angle, play_pattern.name, shot.statsbomb_xg,
         pressure, first_time, aerial_won, one_on_one, obstruction,
         GKpos, GKdist, shot.type.name, shot.technique.name,
         shot.body_part.name, assist.aerial, assist.cross,
         assist.switch, assist.through, assist.cut_back,
         assist.inswinging, assist.outswinging, assist.deflected,
         outcome)
```

```
  return(shots)
}
```

Dataset Creation

This chunk of code is responsible for creating the entire datasets. Because the goal of the project is to analyze the predictive performance of my models as compared to a professional model, we need training and testing sets. Thus the data is partitioned into an 80/20 split and two datasets are created.

```
comp <- FreeCompetitions()
matches <- FreeMatches(comp)

index <- createDataPartition(y = matches$match_id, p = 0.8, list = FALSE)
train_matches <- matches[index, ]
test_matches <- matches[-index, ]

training <- getnewShots(train_matches[1, ])

for (l1 in 2:nrow(train_matches)) {
  tr2 <- getnewShots(train_matches[l1, ])
  training <- training |> bind_rows(tr2)
}

testing <- getnewShots(test_matches[1, ])

for (l2 in 2:nrow(test_matches)) {
  te2 <- getnewShots(test_matches[l2, ])
  testing <- testing |> bind_rows(te2)
}

all_shots <- training |> bind_rows(testing)
training <- training |> select(-shot.statsbomb_xg)
```

Dataset Saving and Loading

Recreating the dataset everytime you want to run the model becomes quite tedious. As such the datasets are saved for future use using this first code chunk.

```
saveRDS(training, file = "training.RDS")
saveRDS(testing, file = "testing.RDS")
saveRDS(test_matches, file = "testMatches.RDS")
saveRDS(train_matches, file = "trainMatches.RDS")
saveRDS(matches, file = "matches.RDS")
saveRDS(comp, file = "comp.RDS")
```

Once saved, the datasets can be accessed at will using the following code chunk. The "training2" and "testing2" datasets are duplicates of the existing datasets with the sole difference being that the "Goal" variable becomes a factor with the levels "Goal" and "NoGoal" as opposed to the binary 0 and 1 in the "training" and "testing" sets.

```r
training <- readRDS("training.RDS")
testing <- readRDS("testing.RDS")
test_matches <- readRDS("testMatches.RDS")
train_matches <- readRDS("trainMatches.RDS")
matches <- readRDS("matches.RDS")
comp <- readRDS("comp.RDS")
training <- training |> filter(!is.na(GKdist) && angle >= 0 && angle <= 90)
testing <- testing |> filter(!is.na(GKdist) && angle >= 0 && angle <= 90)
training2 <- training |> mutate(
  outcome2 = case_when(outcome == 1 ~ "Goal", .default = "NoGoal")
) |> select(-outcome)
testing2 <- testing |> mutate(
  outcome2 = case_when(outcome == 1 ~ "Goal", .default = "NoGoal")
) |> select(-outcome)
```

Model Building

Having created a functional dataset, the actual models have to be built. I will build four different models using four different techniques: logistic regression, LASSO regression, Ridge regression, and XGBoost.

Logistic Regression

Variable Selection

Fitting the logistic regression model has a few steps. First a general, all encompassing model is built. Then, variable selection is performed to find a more accurate and interpretable model. Finally the model is assessed using cross validation.

```r
logVS_mod <- glm(
  outcome ~ . + distance:angle + distance:shot.body_part.name +
    distance:GKdist + angle:obstruction + angle:GKpos +
    GKdist:GKpos + distance:obstruction, data = training,
  family = binomial(link = "logit")
  )
summary(logVS_mod)
```

```
##
## Call:
## glm(formula = outcome ~ . + distance:angle + distance:shot.body_part.name +
##     distance:GKdist + angle:obstruction + angle:GKpos + GKdist:GKpos +
##     distance:obstruction, family = binomial(link = "logit"),
##     data = training)
##
## Coefficients: (7 not defined because of singularities)
##                                        Estimate Std. Error z value Pr(>|z|)
## (Intercept)                           1.1637788  0.6465820   1.800 0.071877 .
## distance                             -0.2226748  0.0164149 -13.565  < 2e-16 ***
## angle                                 0.0416623  0.0020928  19.908  < 2e-16 ***
## play_pattern.nameFrom Counter         0.1465849  0.0753484   1.945 0.051724 .
## play_pattern.nameFrom Free Kick      -0.0069190  0.0581856  -0.119 0.905344
```

```
## play_pattern.nameFrom Goal Kick         0.1324933  0.0825823   1.604 0.108631
## play_pattern.nameFrom Keeper            0.0652470  0.1191552   0.548 0.583980
## play_pattern.nameFrom Kick Off         -0.1426276  0.1487076  -0.959 0.337501
## play_pattern.nameFrom Throw In         -0.0022401  0.0582885  -0.038 0.969344
## play_pattern.nameOther                  0.1246293  0.2556791   0.487 0.625944
## play_pattern.nameRegular Play           0.0729431  0.0540296   1.350 0.176998
## pressureTRUE                           -0.0844930  0.0388141  -2.177 0.029491 *
## first_timeTRUE                          0.1201121  0.0369905   3.247 0.001166 **
## aerial_wonTRUE                         -0.4933149  0.0781128  -6.315 2.69e-10 ***
## one_on_oneTRUE                          0.1654771  0.0518986   3.188 0.001430 **
## obstruction                            -0.2274027  0.0641446  -3.545 0.000392 ***
## GKposTRUE                              -0.3831548  0.1324293  -2.893 0.003812 **
## GKdist                                 -0.0815531  0.0098152  -8.309  < 2e-16 ***
## shot.type.nameFree Kick                -1.8601343  0.5833873  -3.189 0.001430 **
## shot.type.nameKick Off                 -7.2536000 72.4670339  -0.100 0.920269
## shot.type.nameOpen Play                -3.3314129  0.5765206  -5.778 7.54e-09 ***
## shot.technique.nameDiving Header        1.8966718  0.2774480   6.836 8.14e-12 ***
## shot.technique.nameHalf Volley          0.8780396  0.2222576   3.951 7.80e-05 ***
## shot.technique.nameLob                  1.7909823  0.2438996   7.343 2.09e-13 ***
## shot.technique.nameNormal               1.2875532  0.2207900   5.832 5.49e-09 ***
## shot.technique.nameOverhead Kick        0.2373955  0.2963268   0.801 0.423058
## shot.technique.nameVolley               0.8859800  0.2244007   3.948 7.87e-05 ***
## shot.body_part.nameLeft Foot           -0.3317002  0.1218449  -2.722 0.006483 **
## shot.body_part.nameOther                0.5913749  0.4574047   1.293 0.196048
## shot.body_part.nameRight Foot          -0.3286993  0.1176965  -2.793 0.005226 **
## assist.aerialTRUE                      -0.1675552  0.1226745  -1.366 0.171985
## assist.aerialUnassisted                 0.1609239  0.0389413   4.132 3.59e-05 ***
## assist.crossTRUE                       -0.0549615  0.0448648  -1.225 0.220558
## assist.crossUnassisted                         NA         NA      NA       NA
## assist.switchTRUE                      -0.0281343  0.0815861  -0.345 0.730214
## assist.switchUnassisted                        NA         NA      NA       NA
## assist.throughTRUE                      0.5818954  0.0544190  10.693  < 2e-16 ***
## assist.throughUnassisted                       NA         NA      NA       NA
## assist.cut_backTRUE                    -0.0419649  0.0877867  -0.478 0.632627
## assist.cut_backUnassisted                      NA         NA      NA       NA
## assist.inswingingTRUE                  -0.2036027  0.1111571  -1.832 0.067001 .
## assist.inswingingUnassisted                    NA         NA      NA       NA
## assist.outswingingTRUE                 -0.3874569  0.1141168  -3.395 0.000686 ***
## assist.outswingingUnassisted                   NA         NA      NA       NA
## assist.deflectedTRUE                    0.2022271  0.3384578   0.597 0.550176
## assist.deflectedUnassisted                     NA         NA      NA       NA
## distance:angle                         -0.0004950  0.0001395  -3.548 0.000388 ***
## distance:shot.body_part.nameLeft Foot   0.1418165  0.0134000  10.583  < 2e-16 ***
## distance:shot.body_part.nameOther      -0.1329325  0.0783548  -1.697 0.089782 .
## distance:shot.body_part.nameRight Foot  0.1450775  0.0132362  10.961  < 2e-16 ***
## distance:GKdist                         0.0017174  0.0002368   7.251 4.14e-13 ***
## angle:obstruction                      -0.0026509  0.0009718  -2.728 0.006378 **
## angle:GKposTRUE                        -0.0071875  0.0021096  -3.407 0.000657 ***
## GKposTRUE:GKdist                       -0.0264605  0.0087210  -3.034 0.002413 **
## distance:obstruction                    0.0114917  0.0021752   5.283 1.27e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
```

```
##
##     Null deviance: 45093  on 69201  degrees of freedom
## Residual deviance: 36854  on 69154  degrees of freedom
## AIC: 36950
##
## Number of Fisher Scoring iterations: 8
```

```r
logVS <- stepAIC(object = logVS_mod)
```

```r
log_mod <- glm(logVS$formula,
    family = binomial(link = "logit"), data = training)
summary(log_mod)
```

```
##
## Call:
## glm(formula = logVS$formula, family = binomial(link = "logit"),
##     data = training)
##
## Coefficients: (2 not defined because of singularities)
##                                          Estimate Std. Error z value Pr(>|z|)
## (Intercept)                               1.0686357  0.6443953   1.658 0.097246 .
## distance                                 -0.2179927  0.0162723 -13.397  < 2e-16 ***
## angle                                     0.0414743  0.0020851  19.890  < 2e-16 ***
## pressureTRUE                             -0.0856523  0.0387941  -2.208 0.027254 *
## first_timeTRUE                            0.1081864  0.0364518   2.968 0.002998 **
## aerial_wonTRUE                           -0.4965990  0.0780191  -6.365 1.95e-10 ***
## one_on_oneTRUE                            0.1739832  0.0517311   3.363 0.000770 ***
## obstruction                              -0.2403542  0.0639273  -3.760 0.000170 ***
## GKposTRUE                                -0.3805857  0.1323014  -2.877 0.004019 **
## GKdist                                   -0.0816443  0.0098053  -8.327  < 2e-16 ***
## shot.type.nameFree Kick                  -1.8542955  0.5799038  -3.198 0.001386 **
## shot.type.nameKick Off                   -7.3069000 72.4668802  -0.101 0.919685
## shot.type.nameOpen Play                  -3.2785408  0.5745335  -5.706 1.15e-08 ***
## shot.technique.nameDiving Header          1.9224244  0.2769280   6.942 3.87e-12 ***
## shot.technique.nameHalf Volley            0.8784314  0.2219131   3.958 7.54e-05 ***
## shot.technique.nameLob                    1.8071981  0.2434909   7.422 1.15e-13 ***
## shot.technique.nameNormal                 1.3096180  0.2202857   5.945 2.76e-09 ***
## shot.technique.nameOverhead Kick          0.2229878  0.2959189   0.754 0.451123
## shot.technique.nameVolley                 0.8799102  0.2241172   3.926 8.63e-05 ***
## shot.body_part.nameLeft Foot             -0.2834389  0.1198117  -2.366 0.017996 *
## shot.body_part.nameOther                  0.6067985  0.4571358   1.327 0.184379
## shot.body_part.nameRight Foot            -0.2817281  0.1156721  -2.436 0.014868 *
## assist.throughTRUE                        0.6039042  0.0533928  11.311  < 2e-16 ***
## assist.throughUnassisted                  0.1814713  0.0354814   5.115 3.14e-07 ***
## assist.inswingingTRUE                    -0.1886429  0.0979259  -1.926 0.054056 .
## assist.inswingingUnassisted                      NA         NA      NA       NA
## assist.outswingingTRUE                   -0.3743010  0.1019554  -3.671 0.000241 ***
## assist.outswingingUnassisted                     NA         NA      NA       NA
## distance:angle                           -0.0004970  0.0001394  -3.564 0.000365 ***
## distance:shot.body_part.nameLeft Foot     0.1414452  0.0133524  10.593  < 2e-16 ***
## distance:shot.body_part.nameOther        -0.1337840  0.0783798  -1.707 0.087847 .
## distance:shot.body_part.nameRight Foot    0.1447916  0.0131899  10.977  < 2e-16 ***
## distance:GKdist                           0.0016435  0.0002371   6.933 4.13e-12 ***
```

```
## angle:obstruction                      -0.0025673  0.0009717  -2.642 0.008241 **
## angle:GKposTRUE                        -0.0073765  0.0021058  -3.503 0.000460 ***
## GKposTRUE:GKdist                       -0.0270136  0.0087080  -3.102 0.001921 **
## distance:obstruction                    0.0115049  0.0021666   5.310 1.10e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 45093  on 69201  degrees of freedom
## Residual deviance: 36872  on 69167  degrees of freedom
## AIC: 36942
##
## Number of Fisher Scoring iterations: 8
```

Cross Validation

```r
five_folds <- trainControl(
  method="cv", number=5,
  summaryFunction = twoClassSummary,
  classProbs = TRUE
  )

training2$outcome2 <- factor(training2$outcome2)

train(outcome2 ~ distance + angle + pressure + first_time + aerial_won +
    one_on_one + obstruction + GKpos + GKdist + shot.type.name +
    shot.technique.name + shot.body_part.name + assist.aerial +
    assist.through + assist.inswinging + assist.outswinging +
    distance:angle + distance:shot.body_part.name + distance:GKdist +
    angle:GKpos + GKpos:GKdist + distance:obstruction,
  data = training2,
  trControl = five_folds,
  method = "glm",
  family = binomial(link = "logit"),
  metric = "ROC"
)
```

```
## Generalized Linear Model
##
## 69202 samples
##    16 predictor
##     2 classes: 'Goal', 'NoGoal'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 55362, 55362, 55361, 55362, 55361
## Resampling results:
##
##   ROC        Sens       Spec
##   0.7981454  0.1111917  0.9916317
```

Penalized Regression

The process for both LASSO and Ridge regression is similar to the logistic regression process, but there are a few differences. First, both types of regression require a specific lambda value. The optimal lambda value can be found using cross validation. The other difference is that there is no need for variable selection because of the penalization process internal to the two regressions.

LASSO Regression

```
lasData <- training |> select(-outcome)

lasData <- lasData |> rowwise() |> mutate(
  "d:a" = distance*angle,
  "d:gd" = distance*GKdist,
  "a:gp" = angle*GKpos,
  "gp:gd" = GKpos*GKdist,
  "d:ob" = distance*obstruction,
  "d:h" = case_when(shot.body_part.name == "Head" ~ distance, .default = 0),
  "d:lf" = case_when(shot.body_part.name == "Left Foot" ~ distance, .default = 0),
  "d:rf" = case_when(shot.body_part.name == "Right Foot" ~ distance, .default = 0),
  "d:o" = case_when(shot.body_part.name == "Other" ~ distance, .default = 0)
) |> ungroup() |> data.matrix()

find_lasLambda <- cv.glmnet(lasData, training$outcome, alpha = 1)
lasLambda <- find_lasLambda$lambda.min
las_mod <- glmnet(lasData, training$outcome, alpha = 1, lambda = lasLambda)
coef(las_mod)
```

```
## 31 x 1 sparse Matrix of class "dgCMatrix"
##                                 s0
## (Intercept)          0.3594019266
## distance            -0.0069497901
## angle                0.0071337620
## play_pattern.name    0.0003271630
## pressure            -0.0110806145
## first_time           0.0010764792
## aerial_won          -0.0436686567
## one_on_one           0.0395106679
## obstruction         -0.0450043066
## GKpos               -0.1054983974
## GKdist              -0.0140486293
## shot.type.name      -0.0352079674
## shot.technique.name  0.0028235064
## shot.body_part.name -0.0004481176
## assist.aerial       -0.0264812980
## assist.cross        -0.0179079021
## assist.switch        .
## assist.through       0.1073372111
## assist.cut_back      0.0021895694
## assist.inswinging   -0.0185021480
## assist.outswinging  -0.0402616469
## assist.deflected    -0.0016713797
```

```
## d:a                  -0.0001325149
## d:gd                  0.0003509540
## a:gp                 -0.0025024397
## gp:gd                 0.0041009637
## d:ob                  0.0015056147
## d:h                  -0.0089349979
## d:lf                 -0.0000190142
## d:rf                  0.0002338812
## d:o                  -0.0094113924
```

Ridge Regression

```r
# ridData <- training |> select(-outcome) |> data.matrix()
find_ridLambda <- cv.glmnet(lasData, training$outcome, alpha = 0)
ridLambda <- find_ridLambda$lambda.min
rid_mod <- glmnet(lasData, training$outcome, alpha = 0, lambda = ridLambda)
coef(rid_mod)
```

```
## 31 x 1 sparse Matrix of class "dgCMatrix"
##                                 s0
## (Intercept)           4.144346e-01
## distance             -4.334628e-03
## angle                 4.505654e-03
## play_pattern.name     3.490531e-04
## pressure             -1.232213e-02
## first_time            5.248268e-03
## aerial_won           -4.083992e-02
## one_on_one            4.831088e-02
## obstruction          -3.351000e-02
## GKpos                -1.259189e-01
## GKdist               -7.968089e-03
## shot.type.name       -3.929468e-02
## shot.technique.name   2.744909e-03
## shot.body_part.name -1.288372e-03
## assist.aerial        -1.400227e-02
## assist.cross         -7.169846e-03
## assist.switch        -7.755199e-03
## assist.through        6.543292e-02
## assist.cut_back      -2.371820e-03
## assist.inswinging    -6.609137e-03
## assist.outswinging   -1.887911e-02
## assist.deflected     -3.674927e-03
## d:a                  -6.179692e-05
## d:gd                  1.993961e-04
## a:gp                 -1.213999e-03
## gp:gd                -1.047643e-03
## d:ob                  8.753424e-04
## d:h                  -9.467138e-03
## d:lf                 -1.326612e-03
## d:rf                 -9.606200e-04
## d:o                  -8.396415e-03
```

XGBoost

```
boostData <- list(
  data = select(training, -outcome) |> data.matrix(),
  label = training$outcome |> as.factor()
  )

boost_mod <- xgboost(
  x = as.matrix(boostData$data),
  y = boostData$label,
  max_depth = 2,
  learning_rate = 1,
  nrounds = 2,
  objective = "binary:logistic",
  verbosity = 1
  )
```

```
## [1]  train-logloss:0.302819
## [2]  train-logloss:0.287673
```

Testing

There are a few different ways of testing these models. One is a comparison of the model-generated xG values to the Statsbomb-generated xG values. The other is a comparison of all the xG values to the actual shot outcome.

Model Predictions

In order to make these comparisons about the predictive capabilities, we must first use the developed models to make predictions. This is done by using all four of the models described above to estimate the xG of every shot in the "testing" dataset.

```
log_pred <- predict(log_mod, newdata = testing, type = "response")

pred_regX <- testing |>
  select(-outcome, -shot.statsbomb_xg)|>
  rowwise() |> mutate(
    "d:a" = distance*angle,
    "d:gd" = distance*GKdist,
    "a:gp" = angle*GKpos,
    "gp:gd" = GKpos*GKdist,
    "d:ob" = distance*obstruction,
    "d:h" = case_when(shot.body_part.name == "Head" ~ distance, .default = 0),
    "d:lf" = case_when(shot.body_part.name == "Left Foot" ~ distance, .default = 0),
  "d:rf" = case_when(shot.body_part.name == "Right Foot" ~ distance, .default = 0),
  "d:o" = case_when(shot.body_part.name == "Other" ~ distance, .default = 0)
    ) |> ungroup() |> data.matrix()

las_pred <- predict(las_mod, newx = pred_regX, type = "response")
```

```r
rid_pred <- predict(rid_mod, newx = pred_regX, type = "response")

boost_pred <- predict(boost_mod, newdata = testing |> select(-outcome, -shot.statsbomb_xg))

testing <- testing |> ungroup() |> mutate(
  LogXG = log_pred,
  LasXG = las_pred,
  RidXG = rid_pred,
  BoostXG = boost_pred,
  .after = shot.statsbomb_xg
)
```

Comparison to Professional Model

After finding the models' xG estimation, they are compared to the available Statsbomb xG for every given shot. Summary statistics for these differences are printed below.

```r
testing <- testing |> mutate(
  DifLog = abs(shot.statsbomb_xg - LogXG),
  DifLas = abs(shot.statsbomb_xg - LasXG),
  DifRid = abs(shot.statsbomb_xg - RidXG),
  DifBoost = abs(shot.statsbomb_xg - BoostXG),
  .after = BoostXG
)

compDat <- testing |>
  pivot_longer(cols = c(DifLog, DifBoost, DifLas, DifRid),
               names_to = "Model",
               values_to = "Dif"
               ) |> select(Model, Dif)

compDat |> group_by(Model) |> summarise(
  min = min(Dif),
  Q1 = quantile(Dif, probs = 0.25),
  med = median(Dif),
  avg = mean(Dif),
  Q3 = quantile(Dif, probs = 0.75),
  max = max(Dif)
)
```

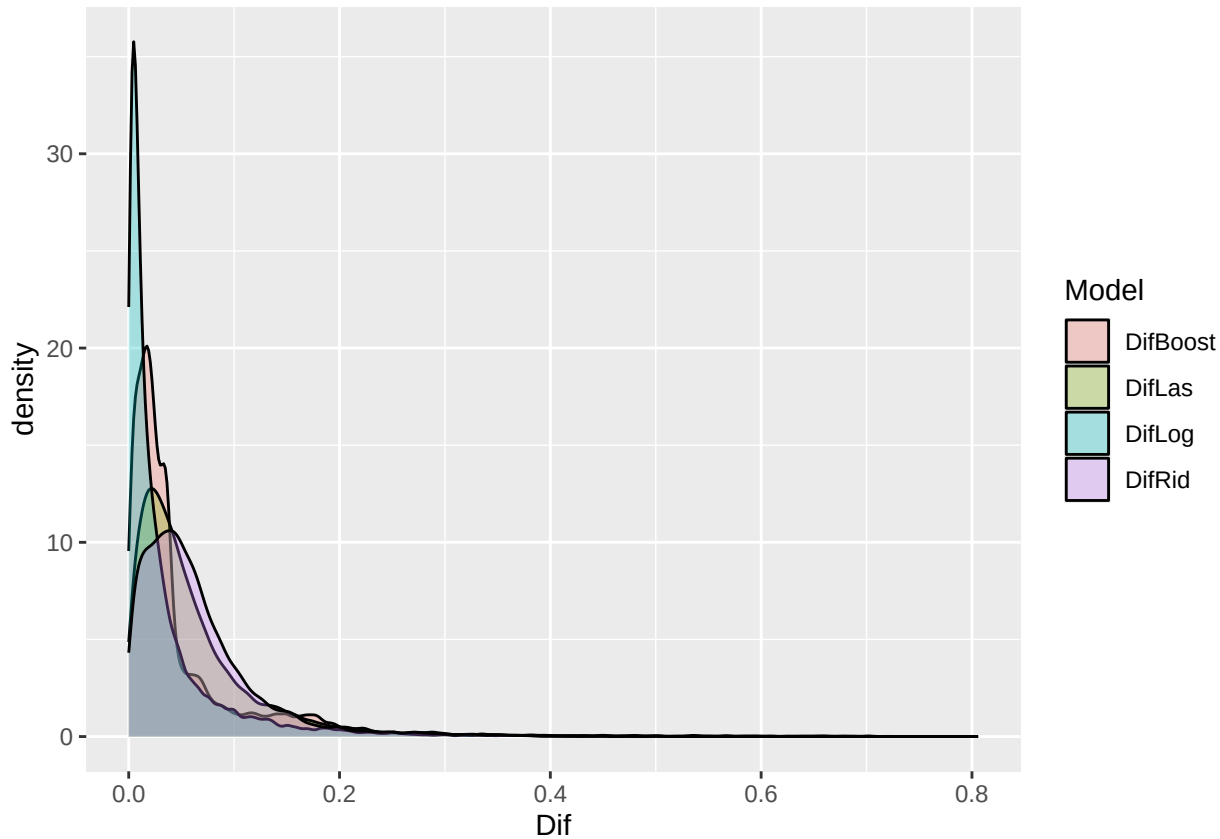
```
## # A tibble: 4 x 7
##   Model            min      Q1    med    avg     Q3   max
##   <chr>          <dbl>   <dbl>  <dbl>  <dbl>  <dbl> <dbl>
## 1 DifBoost 0.00000960  0.0135  0.0262 0.0533 0.0552 0.723
## 2 DifLas   0.00000449  0.0222  0.0433 0.0603 0.0773 0.806
## 3 DifLog   0.000000354 0.00560 0.0155 0.0361 0.0400 0.784
## 4 DifRid   0.000000264 0.0268  0.0505 0.0646 0.0829 0.686
```

After looking at these summary statistics, it can be seen that the logistic regression model performs the best by the majority of the metrics. 75% of the its predictions are within 0.04 of the Statsbomb's predictions. The concession here is that the maximum difference, for all the models, is poor. This suggests that there is some element of the shot captured in the data that Statsbomb uses that I was not able to capture here. While unfortunate, the overwhelming majority of the models predictions perform essentially exactly as expected.

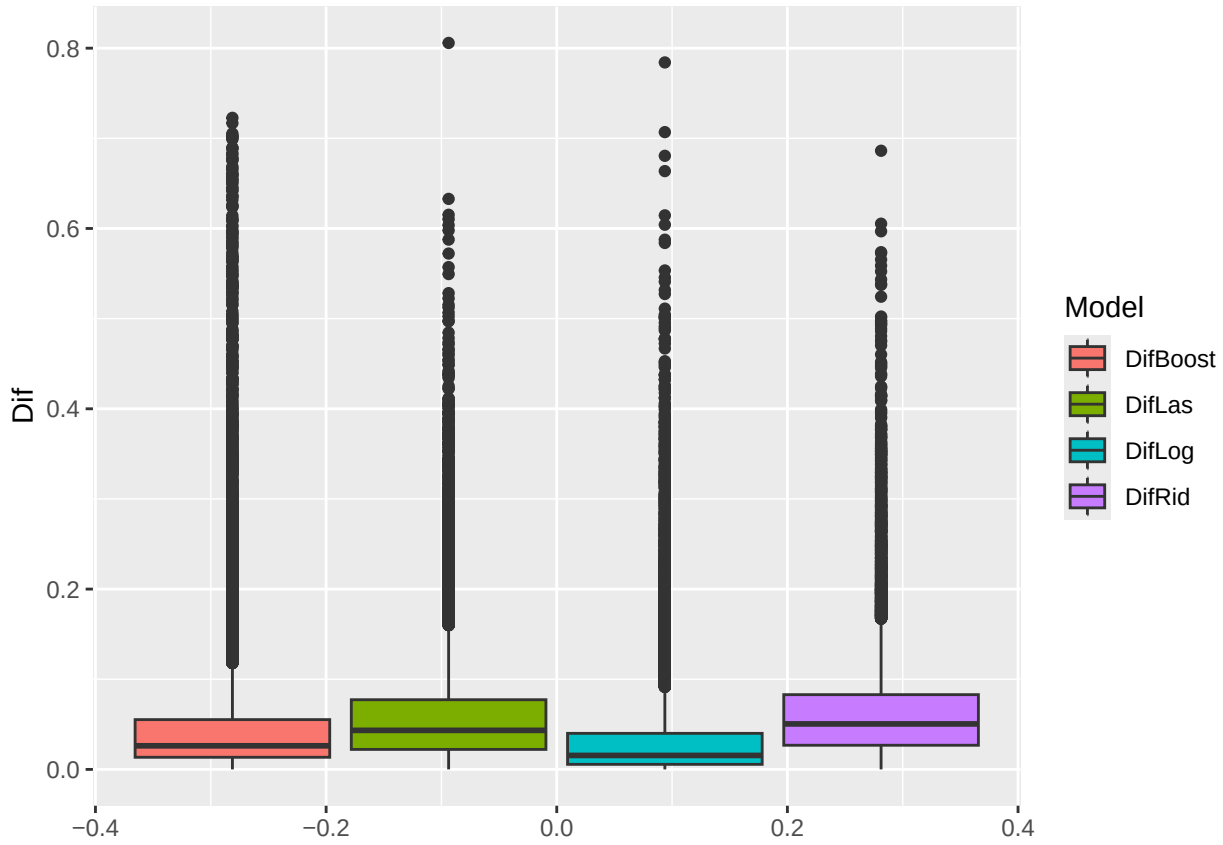
Visualization

To better understand how the models acted in comparison to Statsbomb, here are a few simple visualizations. The first shows the density of differences between. You can see in this one that the peak of the difference associated with the logistic model is the leftmost and the highest. The second graph is a visualization of the summary statistics displayed above.

```
ggplot(compDat, aes(x= Dif, fill = Model)) +
  geom_density(alpha = 0.3)
```




```
ggplot(compDat, aes(y= Dif, fill = Model)) +
  geom_boxplot()
```



Comparison to Shot Outcome

Another way of checking the results of these models is comparing the predictions to the actual outcome of the shots. The following code generates a 1, 0, or -1 for every shot. If the shot ended in a goal but the model predicted no goal, it will be a 1. If the model was accurate in predicting either goal or no goal, it will be a 0. If the shot did not result in a goal but the model predicted a goal, it will be a -1.

```
shotOutcome <- function(outcome, pred) {
  #true neg
  if (outcome == 0 && pred == 0) {
    return("trueNeg")
  }
  #true pos
  if (outcome == 1 && pred == 1) {
    return("truePos")
  }
  #false pos
  if (outcome == 0 && pred == 1) {
    return("falsePos")
  }
  #false neg
  if (outcome == 1 && pred == 0) {
    return("falseNeg")
  }
}
```

```r
countDat <- testing |> rowwise() |> mutate(
  LogPred = case_when(LogXG >= 0.5 ~ 1, .default = 0),
  SBPred = case_when(shot.statsbomb_xg >= 0.5 ~ 1, .default = 0),
  LasPred = case_when(LasXG >= 0.5 ~ 1, .default = 0),
  RidPred = case_when(RidXG >= 0.5 ~ 1, .default = 0),
  BoostPred = case_when(BoostXG >= 0.5 ~ 1, .default = 0),
  BoostPredDif = shotOutcome(outcome, BoostPred),
  LasPredDif = shotOutcome(outcome, LasPred),
  RidPredDif = shotOutcome(outcome, RidPred),
  LogPredDif = shotOutcome(outcome, LogPred),
  SBPredDif = shotOutcome(outcome, SBPred)
) |> select(BoostPredDif, LasPredDif, RidPredDif, LogPredDif, SBPredDif)
```

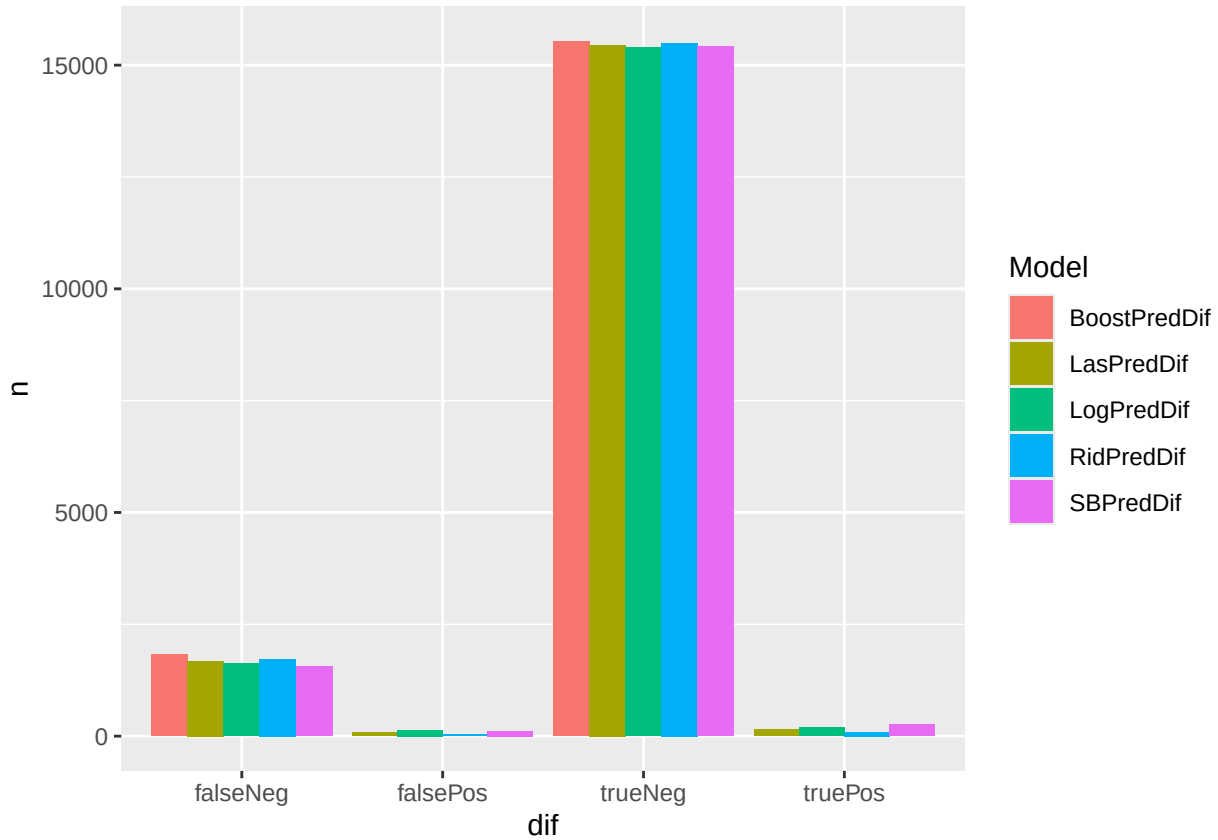
Here you can see the efficacy of each model.

```r
countDat <- countDat |>
  pivot_longer(everything(),
               names_to = "Model",
               values_to = "dif") |>
  count(Model, dif)

disp_table <- countDat |>
  pivot_wider(names_from = dif,
              values_from = n,
              values_fill = 0) |>
  mutate("true" = truePos + trueNeg)

disp_table
```

```
## # A tibble: 5 x 6
##   Model        falseNeg trueNeg falsePos truePos  true
##   <chr>           <int>   <int>    <int>   <int> <int>
## 1 BoostPredDif     1823   15532        0       0 15532
## 2 LasPredDif       1678   15448       84     145 15593
## 3 LogPredDif       1624   15394      138     199 15593
## 4 RidPredDif       1728   15494       38      95 15589
## 5 SBPredDif        1561   15414      118     262 15676
```

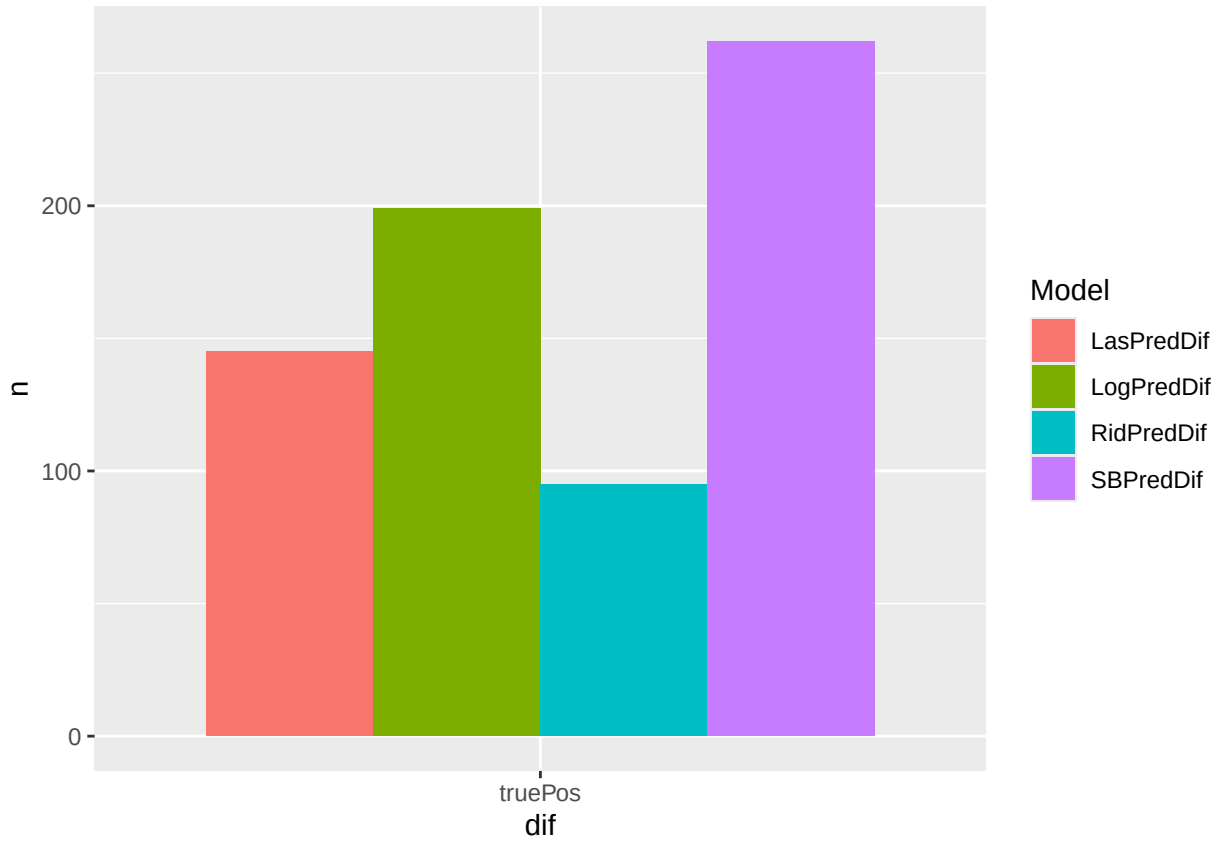
If you graph these results on a single graph, it is hard to see the differences in the number of inaccurate predictions because the scale is vastly different from the accurate predictions.

```r
countDat |> ggplot(aes(x = dif, y = n, fill = Model)) +
  geom_col(position = "dodge")
```

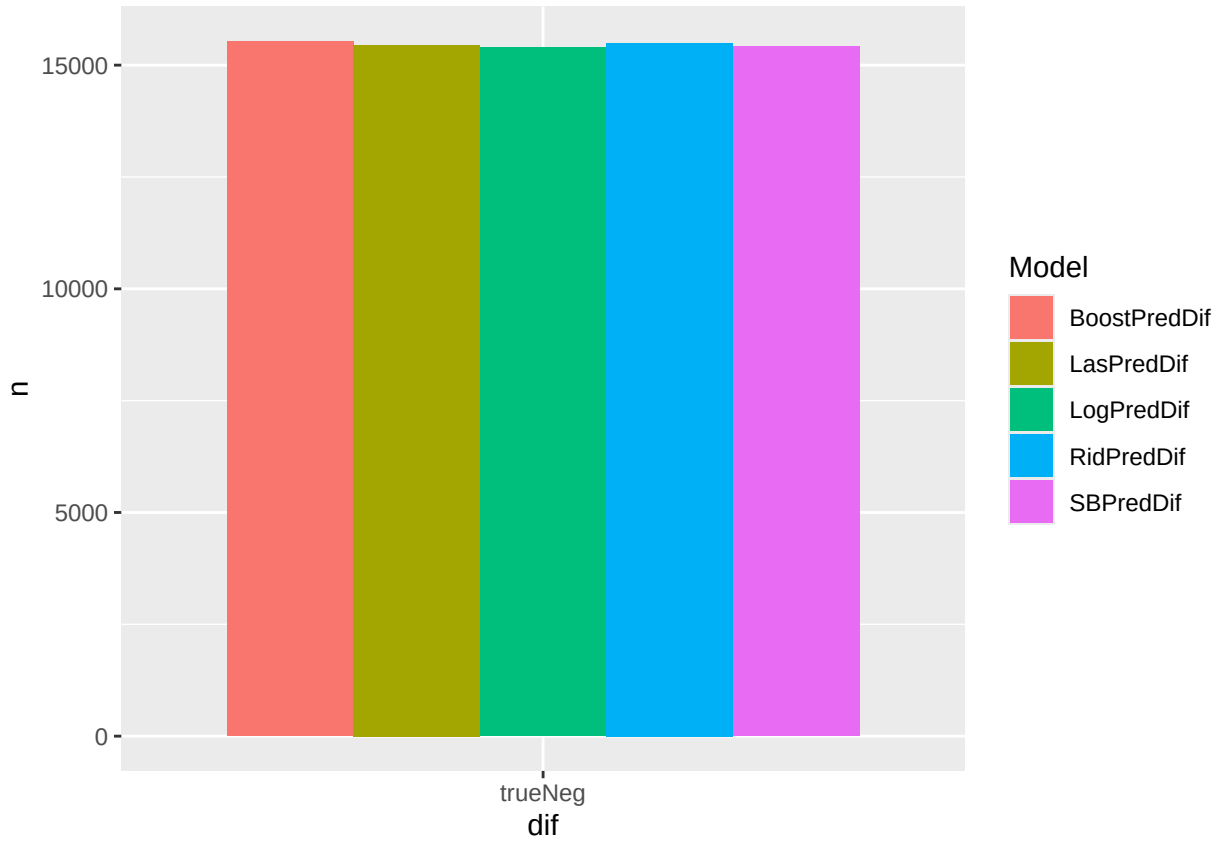


Instead, by breaking up the different cases, the results are much more visually interpretable.

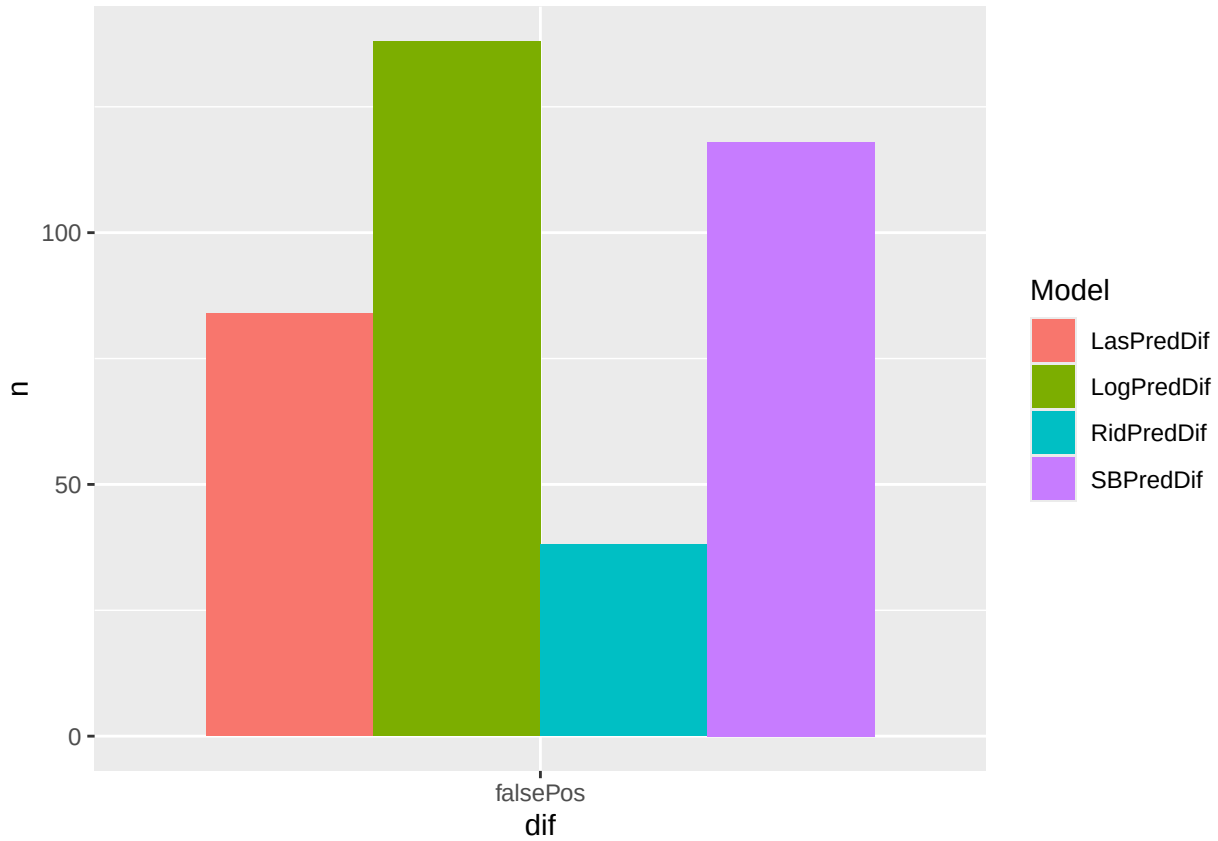
```
countDat |> filter(dif == "truePos") |> ggplot(aes(x = dif, y = n, fill = Model)) +
  geom_col(position = "dodge")
```



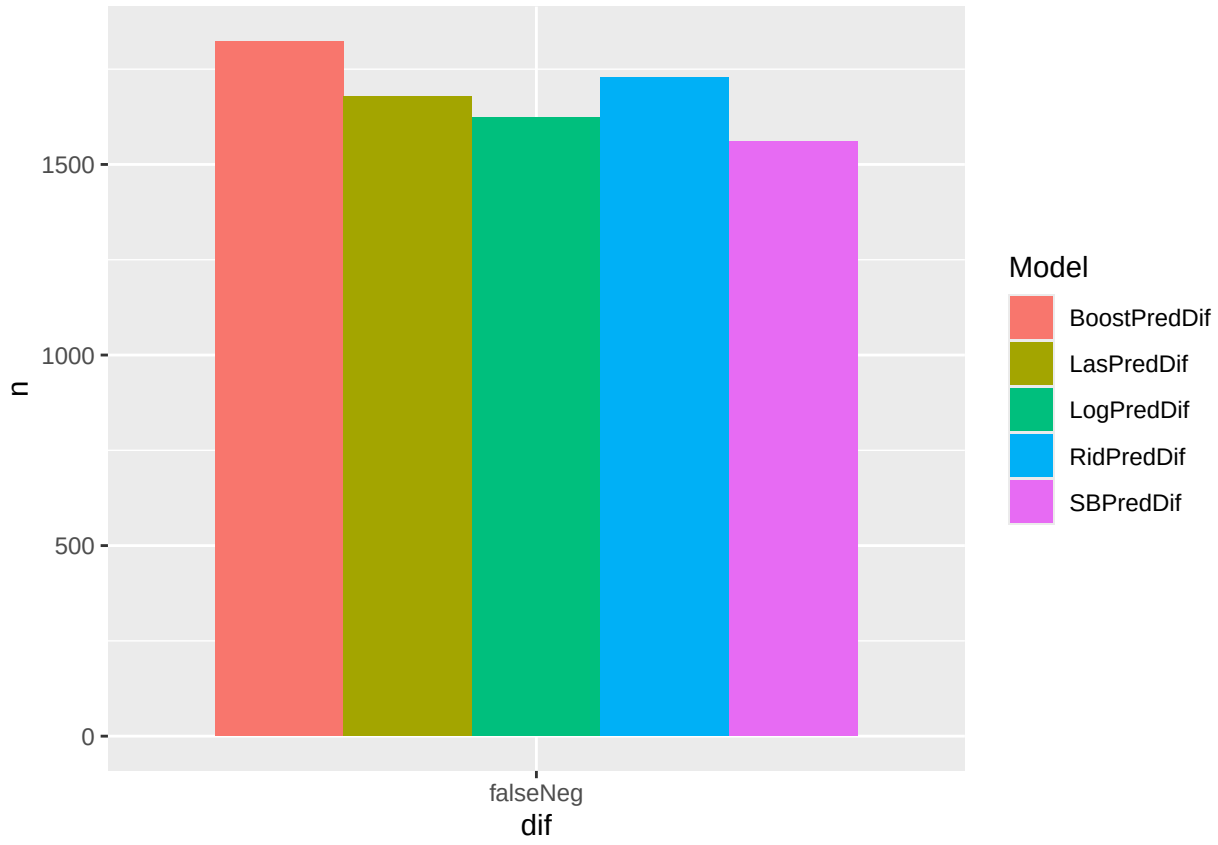

```
countDat |> filter(dif == "trueNeg") |> ggplot(aes(x = dif, y = n, fill = Model)) +
  geom_col(position = "dodge")
```




```
countDat |> filter(dif == "falsePos") |> ggplot(aes(x = dif, y = n, fill = Model)) +
  geom_col(position = "dodge")
```




```
countDat |> filter(dif == "falseNeg") |> ggplot(aes(x = dif, y = n, fill = Model)) +
  geom_col(position = "dodge")
```


1500
1000
500
0
n
falseNeg
dif
Model
BoostPredDif
LasPredDif
LogPredDif
RidPredDif
SBPredDif